Name: Paleru Dharani Govardhan
Reg no : 192525280

TOPIC 1: INTRODUCTION

1. Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.

```
[1]
✓ 0s
    def firstPalindrome(words):
        for word in words:
            if word == word[::-1]:
                return word
        return ""

    print(firstPalindrome(["abc", "car", "ada", "racecar", "cool"]))
```

```
ada
```

2. You are given two integer arrays nums1 and nums2 of sizes n and m, respectively. Calculate the following values: answer1 : the number of indices i such that nums1[i] exists in nums2. answer2 : the number of indices i such that nums2[i] exists in nums1 Return [answer1,answer2].

```
s
    def findIntersectionValues(nums1, nums2):
        set1 = set(nums1)
        set2 = set(nums2)

        answer1 = sum(x in set2 for x in nums1)
        answer2 = sum(x in set1 for x in nums2)

        return [answer1, answer2]

    print(findIntersectionValues([2, 3, 2], [1, 2]))
```

```
[2, 1]
```

3. You are given a 0-indexed integer array nums. The distinct count of a subarray of nums is defined as: Let nums[i..j] be a subarray of nums consisting of all the indices from i to j such that 0 <= i <= j < nums.length. Then the number of distinct values in nums[i..j] is called the distinct count of nums[i..j]. Return the sum of the squares of distinct counts of all subarrays of nums. A subarray is a contiguous non-empty sequence of elements within an array.

[3] ✓ 0s

```
def sumCounts(nums):
    total = 0

    for i in range(len(nums)):
        s = set()
        for j in range(i, len(nums)):
            s.add(nums[j])
            total += len(s) ** 2

    return total

print(sumCounts([1, 2, 1]))
print(sumCounts([1, 1]))
```

```
15
3
```

4. Given a 0-indexed integer array nums of length n and an integer k, return the number of pairs (i, j) where 0 <= i < j < n, such that nums[i] == nums[j] and (i * j) is divisible by k.

[4] ✓ 0s

```
def countPairs(nums, k):
    count = 0

    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] == nums[j] and (i * j) % k == 0:
                count += 1

    return count

print(countPairs([3, 1, 2, 2, 2, 1, 3], 2))
```

```
4
```

5. Write a program FOR THE BELOW TEST CASES with least time complexity

Test Cases: -

1) Input: {1, 2, 3, 4, 5} Expected Output: 5

2) Input: {7, 7, 7, 7, 7} Expected Output: 7

3) Input: {-10, 2, 3, -4, 5} Expected Output: 5

```
[5]
✓ 0s
def find_max(arr):
    if not arr:
        return None

    maximum = arr[0]

    for x in arr:
        if x > maximum:
            maximum = x

    return maximum

print(find_max([1, 2, 3, 4, 5]))
print(find_max([7, 7, 7, 7, 7]))
print(find_max([-10, 2, 3, -4, 5]))

5
7
5
```

6. You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

```
[6]
✓ 0s
def find_max_sorted(arr):
    if not arr:
        return None

    arr.sort()
    return arr[-1]

print(find_max_sorted([]))
print(find_max_sorted([5]))
print(find_max_sorted([3, 3, 3, 3, 3]))

None
5
3
```

Test Cases

1. Empty List

1. Input: []

7. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space

complexity of the algorithm?

```
def unique_elements(arr):
    return list(dict.fromkeys(arr))

print(unique_elements([3, 7, 3, 5, 2, 5, 9, 2]))
print(unique_elements([-1, 2, -1, 3, 2, -2]))
print(unique_elements([1000000, 999999, 1000000]))
```

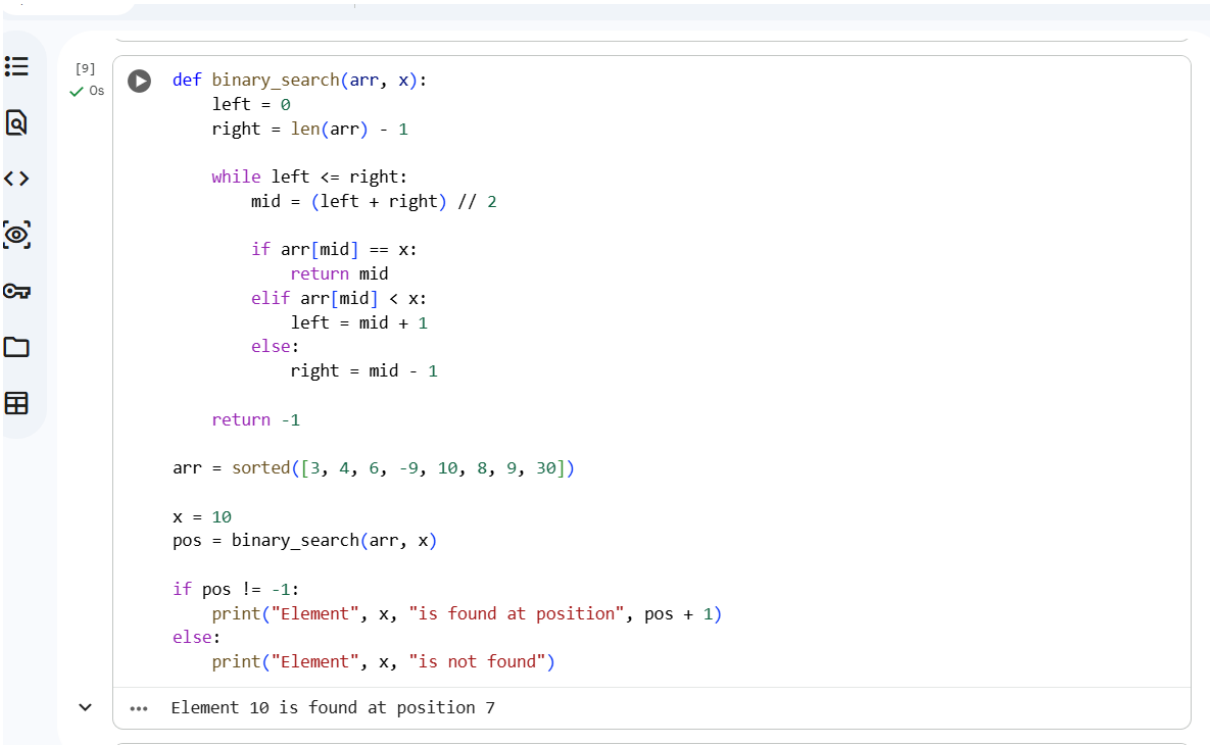
```
[3, 7, 5, 2, 9]
[-1, 2, 3, -2]
[1000000, 999999]
```

8. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        swapped = False

        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        if not swapped:
            break

    return arr

print(bubble_sort([5, 3, 8, 4, 2]))
```

```
[2, 3, 4, 5, 8]
```

9. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

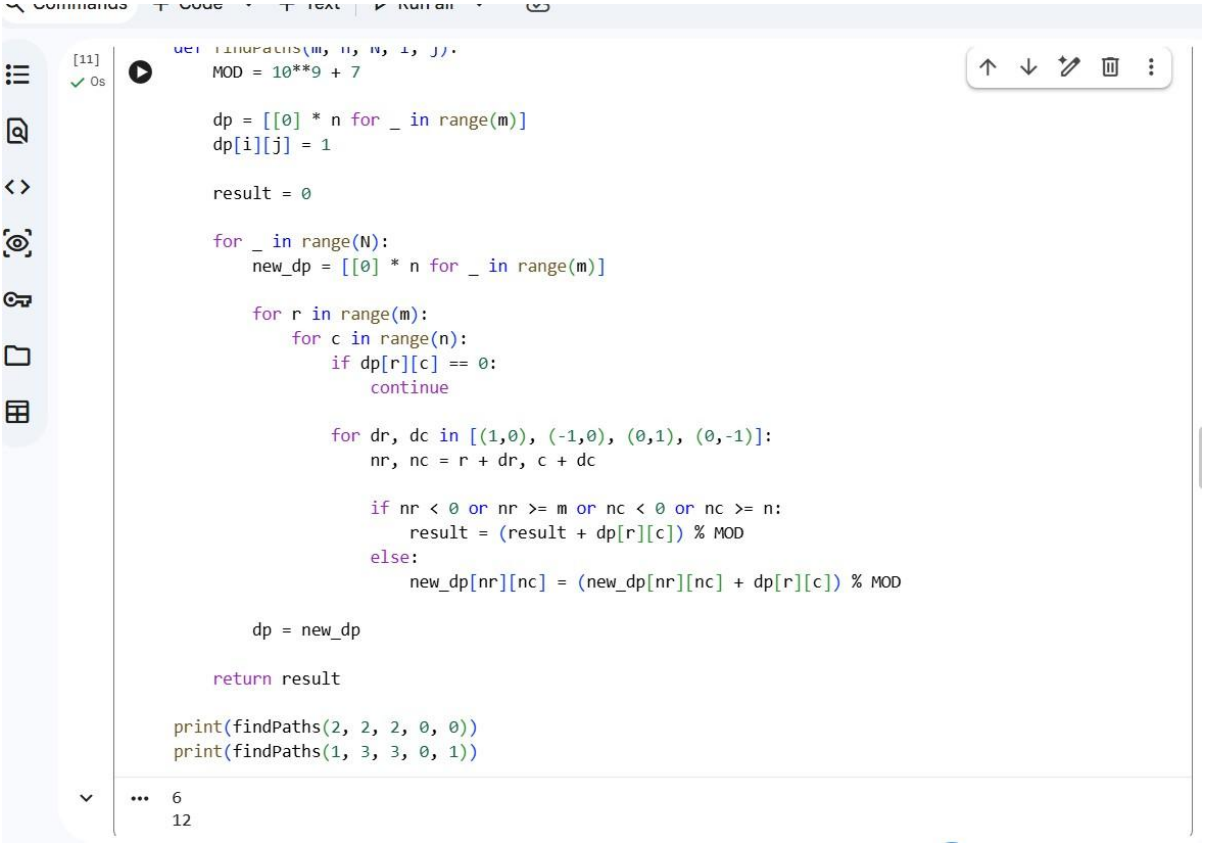


```
[9]
✓ 0s

def binary_search(arr, x):
    left = 0
    right = len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == x:
            return mid
        elif arr[mid] < x:
            left = mid + 1
        else:
            right = mid - 1

    return -1

arr = sorted([3, 4, 6, -9, 10, 8, 9, 30])

x = 10
pos = binary_search(arr, x)

if pos != -1:
    print("Element", x, "is found at position", pos + 1)
else:
    print("Element", x, "is not found")

...  Element 10 is found at position 7
```

10. Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in O(nlog(n)) time complexity and with the smallest space complexity possible.

```
[10]
✓ 0s

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result

print(merge_sort([5, 2, 8, 1, 3]))

...  [1, 2, 3, 5, 8]
```

11. Given an m x n grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps.



```
[11]
✓ 0s
    MOD = 10**9 + 7

    dp = [[0] * n for _ in range(m)]
    dp[i][j] = 1

    result = 0

    for _ in range(N):
        new_dp = [[0] * n for _ in range(m)]

        for r in range(m):
            for c in range(n):
                if dp[r][c] == 0:
                    continue

                for dr, dc in [(1,0), (-1,0), (0,1), (0,-1)]:
                    nr, nc = r + dr, c + dc

                    if nr < 0 or nr >= m or nc < 0 or nc >= n:
                        result = (result + dp[r][c]) % MOD
                    else:
                        new_dp[nr][nc] = (new_dp[nr][nc] + dp[r][c]) % MOD

        dp = new_dp

    return result

print(findPaths(2, 2, 2, 0, 0))
print(findPaths(1, 3, 3, 0, 1))

...
6
12
```

12. You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically

contact the police if two adjacent houses were broken into on the same night.

```
[12]
✓ 0s
def rob(nums):
    if len(nums) == 1:
        return nums[0]

    def rob_linear(arr):
        prev = 0
        curr = 0

        for money in arr:
            prev, curr = curr, max(curr, prev + money)

        return curr

    return max(
        rob_linear(nums[:-1]),
        rob_linear(nums[1:])
    )

print(rob([2, 3, 2]))
print(rob([1, 2, 3, 1]))

3
4
```

13. You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Examples:

(i) Input: n=4 Output: 5

(ii) Input: n=3 Output: 3

```
[13]
✓ 0s
def climbStairs(n):
    if n <= 2:
        return n

    a, b = 1, 2

    for _ in range(3, n + 1):
        a, b = b, a + b

    return b

print(climbStairs(4))
print(climbStairs(3))

5
3
```

14. A robot is located at the top-left corner of a m×n grid .The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there?

Examples:

(i) Input: m=7,n=3 Output: 28

(ii) Input: m=3,n=2 Output: 3

```
[14]
✓ 0s
def uniquePaths(m, n):
    dp = [1] * n

    for _ in range(1, m):
        for j in range(1, n):
            dp[j] += dp[j - 1]

    return dp[-1]

print(uniquePaths(7, 3))
print(uniquePaths(3, 2))
```

```
28
3
```

15. In a string S of lowercase letters, these letters form consecutive groups of the same character. For example, a string like s = "abbxxxxzyy" has the groups "a", "bb", "xxxx", "z", and "yy". A group is identified by an interval [start, end], where start and end denote he start and end indices (inclusive) of the group. In the above example, "xxxx" has the interval [3,6]. A group is considered large if it has 3 or more characters. Return the intervals of every large group sorted in increasing order by start index.

```
[15]
✓ 0s
def largeGroupPositions(s):
    result = []
    start = 0

    for i in range(1, len(s) + 1):
        if i == len(s) or s[i] != s[start]:
            if i - start >= 3:
                result.append([start, i - 1])

            start = i

    return result

print(largeGroupPositions("abbxxxxzzy"))
print(largeGroupPositions("abc"))
```

```
[[3, 6]]
[]
```

16. "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970." The

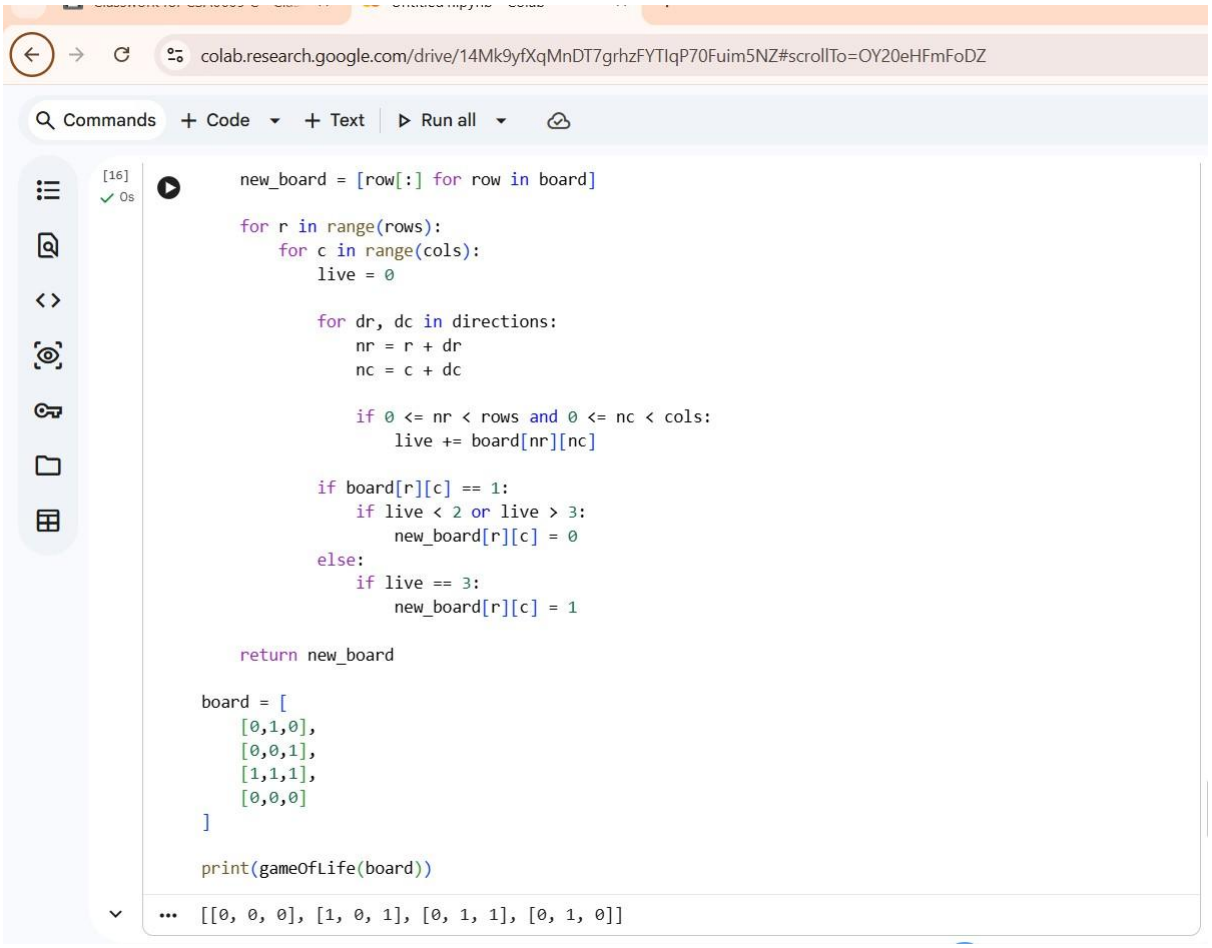
board is made up of an m x n grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules

Any live cell with fewer than two live neighbors dies as if caused by under-population.

1. Any live cell with two or three live neighbors lives on to the next generation.

2. Any live cell with more than three live neighbors dies, as if by over-population.

3. Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction. The next state is created by applying the above rules simultaneously to every cell in the current state, where births and deaths occur .



17. We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2 glasses, and so on until the 100 th row. Each glass holds one cup of champagne. Then, some champagne is poured into the first glass at the top. When the topmost glass is full, any excess liquid poured will fall equally to the glass immediately to the left and right of it. When those glasses become full, any excess champagne will fall equally to the left and right of those glasses, and so on. (A glass at the bottom row has its excess champagne fall on the floor.) For example, after one cup of champagne is poured, the top most glass is full. After two cups of champagne are poured, the two glasses on the second row are half full. After three cups of champagne are poured, those two cups become full - there are 3 full glasses total now. After four cups of champagne are poured, the third row has the middle glass half full, and the two outside glasses are a quarter full, as pictured below. Now after pouring some non-negative integer cups of champagne, return how full the j th glass in the i th row is (both i and j are 0-indexed.)